{vmt DEV

/ bootcamp
1º edición /

Angular

Bootcamp Tercera Edición

Servicios: Interfaces y consumos

Módulo 3

{vmt DEV
/ bootcamp
1ª edición /

Desglose del módulo

1. Inyección de dependencias
2. Interfaces en TypeScript para modelar la respuesta del API
3. Configuración de HttpClient
4. Primer GET contra JSONPlaceholder
5. Mostrado de datos en pantalla

Consumo de un servicio ProductosComponent

```
@Component({...})
export class ProductosComponent {
 productos: any[] =();

 ngOnInit() {

fetch('https://miapi.com/api/v1/productos’)
   .then(res => res.json())
   .then(data => this.productos = data);
}
```

Consumo de un servicio CarritoComponent

```
@Component({...})
export class CarritoComponent {
  productos: any[] = [];

  ngOnInit() {
  fetch('https://mi-
api.com/api/v1/productos’)
  .then(res => res.json())
  .then(data => this.productos = data);
  }
}
```

¿Qué errores hay?

```
@Component({...})
export class CarritoComponent {
  productos: any[] = [];

  ngOnInit() {
  fetch('https://mi-
api.com/api/v1/productos’)
  .then(res => res.json())
  .then(data => this.productos = data);
  }
}
```

¿Qué estaba mal?

1. Recaían todas las funciones en el componente.
2. La URL en el código.
3. El uso del ANY.

```
productos: any[] = [];
this.productos[0].nombree
this.productos[0].preciooo

productos: Producto[] = [];
this.productos[0].nombree
```

1. Creación de un servicio

¿Por qué necesitamos un servicio?

- Hasta ahora toda la lógica vive dentro del componente
- Si dos componentes necesitan los mismos datos, los duplicas
- Los componentes deben encargarse solo de mostrar datos, no de obtenerlos
- Un servicio centraliza la lógica y la comparte entre componentes

Pasos previos

1. El archivo *environment*

```
// src/environments/environment.ts (desarrollo)
export const environment = {
  production: false,
  apiUrl: 'https://jsonplaceholder.typicode.com’
};

// src/environments/environment.prod.ts (producción)
export const environment = {
  production: true,
  apiUrl: 'https://tu-api-real.com/api/v1'
};
```

Pasos previos

2. Configurar HTTP Client

HttpClient es el servicio de Angular para hacer peticiones HTTP.

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners, provideZoneChangeDetection } from
'@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideHttpClient(),
    provideRouter(routes)
  ]
};
```

¿Cómo se crea un servicio?

1. ng generate service productos

```
CREATE src/app/productos.spec.ts (352 bytes)
CREATE src/app/productos.ts (122 bytes)
```

```
1  import { Injectable } from '@angular/core';
2
3  @Injectable({
4    providedIn: 'root',
5  })
6  export class Productos {
7
8  }
9
```

Significa que Angular crea una sola instancia del servicio

{vmt DEV

¿Cómo se crea un servicio?

2. Creación de la interfaz

- Refleja exactamente lo que devuelve el API. Cada objeto anidado es su propia interfaz
- Usa los tipos correctos: string, number, boolean, Tipo[]
- Campos opcionales con ? si el API no siempre los devuelve
- Solo declara los campos que vas a usar
- Nombres en singular y PascalCase: Usuario, no usuarios

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",
      "suite": "Apt. 556",
      "city": "Gwenborough",
      "zipcode": "92998-3874",
      "geo": {
        "lat": "-37.3159",
        "lng": "81.1496"
      }
    },
    "phone": "1-770-736-8031 x56442",
    "website": "hildegard.org",
    "company": {
      "name": "Romaguera-Crona",
      "catchPhrase": "Multi-layered client-server neural-net",
      "bs": "harness real-time e-markets"
    }
  },
```

```
export interface Geo {
  lat: string;
  lng: string;
}

export interface Address {
  street: string;
  suite: string;
  city: string;
  zipcode: string;
  geo: Geo;
}
```

```
export interface Company {
  name: string;
  catchPhrase: string;
  bs: string;
}

export interface Usuario {
  id: number;
  name: string;
  username: string;
  email: string;
  address: Address;
  phone: string;
  website: string;
  company: Company;
}
```

¿Cómo se crea un servicio?

3. Llenado del servicio

El servicio es donde usas *HttpClient* para hacer las peticiones.

- *private apiUrl* viene del *environment*, nunca quemada.
- Cada método devuelve un Observable, no el dato directo.
- El tipo va entre <> para tipar la respuesta.
- El servicio solo hace peticiones, no maneja errores ni loading, eso va en el componente.

```
import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Usuario } from '../interfaces/usuario.interface';
import { environment } from '../../environments/environment';

@Injectable({
  providedIn: 'root'
})
export class UsuariosService {

  private apiUrl = environment.apiUrl;
  private _http = inject(HttpClient);

  getUsuarios(): Observable<Usuario[]> {
    return this._http.get<Usuario[]>(`${this.apiUrl}/users`);
  }
```

```
getUsuarios(): Observable<Usuario[]> {
  return this._http.get<Usuario[]>(`${this.apiUrl}/users`);
}
```

- **Observable<Usuario[]>** El tipo de retorno. Esta función no devuelve usuarios directamente, devuelve un Observable que eventualmente tendrá un arreglo de usuarios. Sin esto TypeScript no sabe qué esperar.

- **return** Devuelve el Observable al componente que lo llamó. El componente luego hace .subscribe() para obtener los datos reales.

- **this._http** El HttpClient inyectado en el constructor.

- **.get** El método HTTP. Le dice a Angular que haga una petición GET.

- **<Usuario[]>** El tipo genérico. Le dice a TypeScript qué forma tiene la respuesta. Sin esto la respuesta sería any.

¿Cómo se crea un servicio?

3. Consumo del servicio en el componente

El servicio es donde usas *HttpClient* para hacer las peticiones.

- *private apiUrl* viene del *environment*, nunca quemada.
- Cada método devuelve un Observable, no el dato directo.
- El tipo va entre <> para tipar la respuesta.
- El servicio solo hace peticiones, no maneja errores ni loading, eso va en el componente.

```
private _usuariosService = inject(UsuariosService);
usuarios: Usuario[] = [];  loading = false;  error =
‘’;
  cargarUsuarios() {
    this.loading = true;
    this.error = '';

    this.usuariosService.getUsuarios().subscribe({
      next: (data) => {
        this.usuarios = data;
        this.loading = false;
      },
      error: (err) => {
        this.error = 'Error al cargar los usuarios';
        this.loading = false;
      }
    });
  }
}
```

```
@if (loading) {
  <div>Cargando...</div>
}

@if (error) {
  <div>{{ error }}</div>
}

@if (!loading && !error) {
  <ul>
    @for (usuario of usuarios; track usuario.id) {
      <li>
        <strong>{{ usuario.name }}</strong> — {{
usuario.email }}
      </li>
    }
  </ul>
}
```

A considerar

- **.subscribe()** Es el momento en que le dices al Observable que obtenga los datos. Hasta que no haces subscribe, el Observable no hace nada, no sale ninguna petición HTTP.

- **next** se ejecuta cuando la petición salió bien y llegaron los datos.

- **error** se ejecuta cuando algo salió mal: no hay internet, el servidor devolvió 404, 500, etc.

{vmt DEV

/ bootcamp
1° edición /

www.vmt.dev

@vmtdev